

More Classifiers and Cross-Validation

The previous chapter used one `train_test_split` to conclude that one model was better. This chapter examines why that conclusion may be unreliable: a single split can simply be lucky or unlucky. The central task is to use cross-validation for a more trustworthy estimate, understand exactly which class is being misclassified, and learn two new classifiers: random forests and K-nearest neighbors.

- Python 3 - scikit-learn
- tutor style
- visual explanations
- FAQ included
- advanced model evaluation

00 Decision Table and Review Order

First identify the evaluation question you are trying to answer. The answer points directly to the right tool.

What you need to do	Recommended tool
A single split feels unreliable; you need a steadier estimate	<code>cross_val_score(model, X, y, cv=5)</code>
A single tree overfits easily	<code>RandomForestClassifier</code> - many trees vote
You have no clear threshold rules and want to classify by similarity	<code>KNeighborsClassifier</code>
You need to know which class the model got wrong	<code>confusion_matrix</code>
Missing a true case is unacceptable, such as disease screening	Focus on <code>recall</code>
A false alarm is unacceptable, such as spam filtering	Focus on <code>precision</code>
You need one complete metric summary	<code>classification_report</code>
Manual hyperparameter trials are too cumbersome	<code>GridSearchCV</code>
You need to compare several models systematically	loop + <code>cross_val_score</code> + DataFrame summary

One evaluation Is it trustworthy?		When it is wrong Which class did it miss?		What matters in this task precision or recall?	
01 - START Why unstable? luck of one split	02 cross_val_score average many splits	03 Random forest many trees vote	04 KNN look at neighbors	05 Confusion matrix wrong-class counts	06 precision / recall two kinds of cost
07 Report all metrics at once	08 GridSearchCV automatic tuning	09 Systematic comparison select the strongest			

The central change in this chapter: instead of asking whether 0.833 is higher than 0.75 and declaring a winner, ask whether the number itself is trustworthy. Model evaluation is like any experiment: one measurement can fluctuate, while repeated measurements and an average are more reliable.

01 Why One train_test_split Is Not Reliable Enough

• Core concept

Change only random_state and accuracy changes

unstable

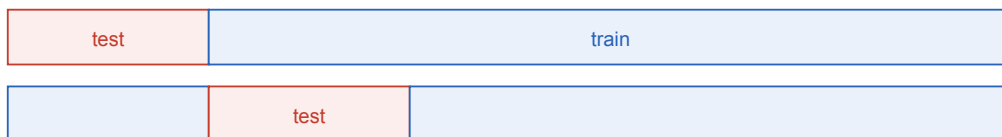
```
for seed in [0, 1, 2, 3, 4]:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=seed
    )
    model = DecisionTreeClassifier(random_state=42)
    model.fit(X_train, y_train)
    print(seed, accuracy_score(y_test, model.predict(X_test)))
# Possible values: 0.83, 0.79, 0.88, 0.75, 0.83
```

Changing only the split seed can move accuracy by several percentage points. The randomness comes from which rows happened to enter the test set: a favorable test set may contain easy examples, while an unlucky one may contain hard examples. A single `train_test_split` accuracy is one sample from a random interval, not the model's fixed, absolute level.

02 cross_val_score: Average Across Repeated Splits

• Core concept

cv=5 divides the data into five folds; each becomes the test fold once



... repeat until every fold has served as test ...

Every row is used once for testing and four times for training. 5 accuracy values -> average -> more reliable estimate

Run cross-validation in one step

cross_val_score

```
scores = cross_val_score(model, X, y, cv=5)
# Run 5 rounds automatically: a different 1/5 is test data each time.
# Return an array containing 5 accuracy values.

scores.mean() # overall performance estimate
scores.std()  # sensitivity to the split; larger means less stable
```

`cv=5` means divide the dataset into five folds. Train five models: each fold becomes the test set once and the other four folds become training data. Every row is tested once and trained on four times. The average accuracy is more trustworthy than one random split; `scores.std()` is also important because large fold-to-fold variation means the model is sensitive to the exact training sample.

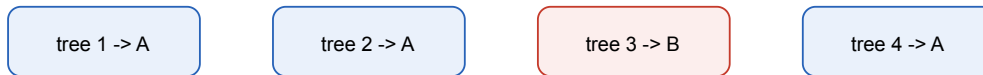
• Common mistake

Incorrect	Why it fails	Correct
Conclude from one <code>train_test_split</code> accuracy	The result is easily moved by split randomness and does not represent stable performance.	Use <code>cross_val_score(model, X, y, cv=5)</code> and average.

03 RandomForestClassifier: Many Trees Vote Together

Core concept

Each tree sees a different sample and subset of candidate features



Majority vote: A appears 3 times, B once -> predict A

A random forest is a committee of diverse decision trees.

An ensemble of many decision trees

forest

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
# Train 100 trees; each sees a different data sample and feature subset.
# The final prediction is the majority vote of those trees.
```

A single decision tree can become attached to peculiarities in a few data points and overfit. A random forest trains many trees on different samples and candidate features, then lets them vote. One tree's bias can be outvoted, so forests are often more stable and less prone to overfitting. `feature_importances_` remains available, now averaged across all trees.

Common mistake

Incorrect

Assume a random forest is always better and switch without comparison

Why it is not ideal

Usually more stable does not mean better on every dataset; it still needs real validation.

Correct

Run the same cross-validation for both models.

04 KNeighborsClassifier: Let the Nearest Neighbors Vote

Core concept

KNN classifies a new point from the labels of its nearest neighbors



KNN learns no explicit rule; prediction depends on nearby labeled training points.

No explicit rule is learned; prediction searches for neighbors

knn

```
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
# k=5: at prediction time, find the five nearest training points.
# The class with the most votes among those five becomes the prediction.
```

KNN thinks differently from trees and forests. A tree learns threshold rules during `fit` and applies them directly during prediction. KNN's `fit` mostly stores the training data; the expensive work happens during `predict`, when distances to all training points are calculated. Because distance is dominated by features with larger numeric ranges, feature scaling is especially important for KNN.

• Common mistake

Incorrect	Why it is risky	Correct
Apply KNN directly when feature scales differ greatly	Large-range variables dominate the distance and drown out other features.	Scale consistently with StandardScaler first; detailed treatment comes later.

05 confusion_matrix: Which Class Did the Model Miss?

• Core concept

		predicted class	
		urgent	normal
true class	urgent	8 correct	2 missed cases
	normal	1 false alarm	15 correct

Diagonal = correct; off-diagonal = the two clinically different error types.

Read the distribution of errors as a matrix confusion

```
confusion_matrix(y_test, model.predict(X_test))
# Rows = true labels; columns = predicted labels.
# The diagonal is correct; the off-diagonal cells are misclassifications.
```

Accuracy only tells you how many predictions were correct. A confusion matrix tells you what the mistakes looked like. At the same 90% accuracy, missing truly urgent patients may be far more dangerous than flagging a normal patient for review. This moves evaluation from one number toward a concrete diagnosis of error.

Q. In a confusion matrix, are rows truth or prediction?

In scikit-learn, rows are the true classes `y_true` and columns are the predicted classes `y_pred`. To find how many truly urgent patients were predicted normal, read the urgent row and normal column. That cell contains missed cases.

06 precision / recall / f1: Two Errors, Two Priorities

• Core concept

Metric	Question it answers	When to care most
precision	Among the cases I called urgent, how many truly were urgent?	False alarms are costly, such as limited resources or spam filtering.
recall	Among all truly urgent cases, how many did I find?	Missed cases are costly, such as disease screening.
f1	What is the harmonic mean of precision and recall?	Both sides matter and one combined score is needed.

Calculate all three metrics for the urgent class precision / recall

```
precision_score(y_test, predictions, pos_label="urgent")
recall_score(y_test, predictions, pos_label="urgent")
f1_score(y_test, predictions, pos_label="urgent")
# pos_label selects the target class that the metric treats as positive.
```

Precision and recall usually trade off. A model that raises many alarms may catch more real cases, increasing recall, while producing more false alarms and lowering precision. A conservative model may do the reverse. Neither is universally better; the right priority depends on which error is more expensive in the real task. F1 is a compromise, not a replacement for domain judgment.

Q. Why does disease screening usually emphasize recall rather than precision?

The costs are asymmetric. A missed case can delay diagnosis and treatment, while a false alarm usually leads to an extra follow-up test. Missing one true disease case is often more costly than investigating several false alarms, so recall deserves greater weight when resources permit.

07

classification_report: View Every Metric at Once

Core concept

Do not calculate every class and metric one by one

report

```
print(classification_report(y_test, predictions))
# For each class: precision, recall, f1-score, and support.
# The bottom also shows overall accuracy and weighted averages.
```

Use `classification_report` when you need a complete summary rather than one hand-calculated metric. It is common in reports and presentations because it prints precision, recall, and F1 for every class at once. The `support` column is easy to overlook but essential: if a class has very few true samples, its metrics can be unstable and should not be overinterpreted.

08

GridSearchCV: Automatically Search Hyperparameters

Core concept

Automate the manual hyperparameter search

grid search

```
param_grid = {
    "max_depth": [2, 4, 6, 8, None],
    "min_samples_split": [2, 5, 10],
}
search = GridSearchCV(
    DecisionTreeClassifier(random_state=42), param_grid, cv=5
)
search.fit(X, y)

search.best_params_ # best-performing parameter combination
search.best_score_  # its mean cross-validation score
```

GridSearchCV automates and expands the manual depth sweep from the previous chapter. It tests every combination in `param_grid`, runs cross-validation for each combination, and reports the highest mean score. As more hyperparameters and candidate values are added, the number of combinations grows quickly, which makes automated search valuable.

Common mistake

Incorrect	Why it fails	Correct
Report <code>search.best_score_</code> as the final test result	The same data was used for parameter search and evaluation, so the result is optimistic.	Use an independent test set, or retain a final untouched holdout set.

Compare Several Models Systematically

Core concept

Loop + cross_val_score + DataFrame summary

[compare](#)

```
models = {
    "decision_tree": DecisionTreeClassifier(random_state=42),
    "random_forest": RandomForestClassifier(
        n_estimators=100, random_state=42
    ),
    "knn": KNeighborsClassifier(n_neighbors=5),
}

rows = []
for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=5)
    rows.append({"model": name, "mean_accuracy": scores.mean(),
                "std": scores.std()})

pd.DataFrame(rows).sort_values("mean_accuracy", ascending=False)
```

This is the chapter's final synthesis. Every model is evaluated with the same cross-validation procedure rather than one train-test split. Results are collected in a DataFrame so the mean estimates can be sorted and variability compared. A regular loop is correct because every iteration trains a completely new model object, not because the same data transformation is being repeated row by row.

Code template

Template - model comparison table

[compare table](#)

```
def compare_models(models_dict, X, y, cv=5):
    rows = []
    for name, model in models_dict.items():
        scores = cross_val_score(model, X, y, cv=cv)
        rows.append({"model": name, "mean_accuracy": scores.mean(),
                    "std": scores.std()})
    return pd.DataFrame(rows).sort_values(
        "mean_accuracy", ascending=False
    )
```

Common Error Summary

Incorrect	Why it fails	Correct
Conclude from one train_test_split result	The result is easily moved by split randomness.	<code>cross_val_score(model, X, y, cv=5)</code>
Assume random forest is always better	Usually more stable does not mean universally better.	Compare with the same cross-validation.
Use KNN without scaling features	Large numeric ranges dominate distance.	Scale first when ranges differ greatly.
Read only accuracy, not the confusion matrix	You cannot tell whether missed cases or false alarms are worse.	Inspect <code>confusion_matrix</code> too.
Always optimize precision or always optimize recall	They trade off, and the right priority depends on error cost.	Decide whether the task fears missed cases or false alarms.
Report <code>search.best_score_</code> as the final score	The same data selected and evaluated the parameters.	Use an untouched test set for final evaluation.

11 FAQ

Q. Do `cross_val_score` and `train_test_split` replace one another?

For evaluating model performance, cross-validation is a more reliable upgrade over one train-test split. When training the final model for use, however, a clear train/test split or a final fit on all approved training data is still common. Cross-validation is used mainly while selecting models and hyperparameters.

Q. Do random-forest and decision-tree `feature_importances_` mean the same thing?

The concept is the same: contribution to the classification task. A random forest averages importance across all trees, which is usually more stable than one tree's ranking. One decision tree may produce a very different ranking after a small data change; averaging reduces that fluctuation.

Q. In a confusion matrix, which axis is truth and which is prediction?

In scikit-learn, rows are true classes and columns are predicted classes. To find missed cases, read the true-positive-class row and the predicted-negative-class column.

Q. Is f1-score always a better choice than precision or recall alone?

No. F1 is the harmonic mean and is useful when both sides matter and one combined number is desired. If the real-world task clearly makes missed cases more costly than false alarms, recall is more direct. If false alarms are more costly, precision is more meaningful.

12 Exercises

• Practice problems

- E** **Verify that one split is unstable:** run `train_test_split` with five different `random_state` values, print each accuracy, and describe the range. Practice interpreting why one evaluation cannot be trusted.
- M** **Compare a decision tree and random forest with cross-validation:** run `cross_val_score(cv=5)` for both models on the same X and y; print each mean and standard deviation. Practice the cross-validation API.
- M** **Interpret a confusion matrix:** given the 2 x 2 matrix shown earlier - urgent truth has 8 predicted urgent and 2 predicted normal, normal truth has 1 predicted urgent and 15 predicted normal - report missed cases and false alarms. Expected: 2 missed cases and 1 false alarm.
- M** **Choose precision or recall:** for disease warning, spam filtering, and face unlock, identify the more important metric and explain why. Practice connecting metric choice to real error costs.
- M** **Compare three models systematically:** use the `compare_models` template for a decision tree, random forest, and `KNN(k=5)`. Return a DataFrame sorted by mean accuracy. Practice integrating all tools from the chapter.

13 Review Order and Chapter Checklist

• Must master

One `train_test_split` accuracy fluctuates and cannot support a direct conclusion

`cross_val_score(model, X, y, cv=5)` averages across folds

`RandomForestClassifier` combines votes from many trees and is usually more stable

`confusion_matrix` uses rows for truth and columns for predictions

precision and recall trade off; neither is universally better

Choose metrics by the cost of false alarms versus missed cases

• Recommended practice

KNeighborsClassifier calculates distances during prediction, unlike tree models

KNN is sensitive to feature scale

classification_report shows all class metrics at once

GridSearchCV automatically searches hyperparameter combinations

Loop + cross_val_score + DataFrame compares several models systematically

• Next steps

Feature scaling: StandardScaler / MinMaxScaler

Pipeline for preprocessing + model

ROC curve and AUC

Regression metrics

More ensembles such as boosting

Deep learning introduction with PyTorch

• Chapter checklist

- ☐ I understand why one train-test accuracy is not the model's true level.
- ☐ I understand how voting across many trees reduces single-tree overfitting risk.
- ☐ I can read matrix rows as truth and columns as predictions.
- ☐ I can choose precision or recall for a concrete task.
- ☐ I understand GridSearchCV as an automated parameter grid plus cross-validation.
- ☐ I can use `cross_val_score(model, X, y, cv=5)` and interpret mean and standard deviation.
- ☐ I understand how KNN prediction differs from tree prediction.
- ☐ I can distinguish missed cases from false alarms.
- ☐ I can obtain all class metrics with `classification_report`.
- ☐ I can build a model-comparison pipeline with a loop, cross-validation, and DataFrame sorting.

After every exercise: you have upgraded from "train one model and calculate one accuracy" to systematic evaluation, comparison, and model selection. This is one of the most time-consuming and most easily overlooked parts of any real machine-learning project. The next step can move into ensembles, feature engineering, or deep learning with PyTorch.